



Security Assessment



Polygon Vault Bridge V1

October 2025

Prepared for Polygon

Table of content

Project Summary.....	3
Project Scope.....	3
Project Overview.....	3
Protocol Overview.....	4
Findings Summary.....	5
Severity Matrix.....	5
Detailed Findings.....	6
Medium Severity Issues.....	8
M-01. NativeConverter is not reconfigurable, breaking upgrade path.....	8
M-02. setCustomToken function is vulnerable to griefing attack.....	9
M-03. Flawed migratable backing calculation prevents fund migration.....	11
Low Severity Issues.....	13
L-01. setYieldVault can lead to temporary insolvency and orphaned assets.....	13
L-02. Deposit DoS risk if USDT enables transfer fees.....	14
Informational Issues.....	15
I-01. Inconsistent validation on destinationNetworkId in VbETH.....	15
I-02. Permissionless reinitializer2 functions are vulnerable to front-running.....	16
I-03. Incorrect rounding in the calculation of migratableGasBacking.....	17
I-04. Missing Events for setBridge and setNativeConverter Functions.....	17
I-05. CustomTokenWethExtension lacks receive/fallback — not WETH9-compatible.....	18
I-06. Unused Error – CannotWrapGasToken().....	18
I-07. Typos.....	19
I-08. README missing a step for Wormhole integration.....	19
Disclaimer.....	21
About Certora.....	21

Project Summary

Project Scope

Project Name	Repository (link)	Latest Commit Hash	Platform
Polygon Vault Bridge V1	https://github.com/agglayer/vault-bridge/tree/feat/v1-dev	3c9d819 (original commit) f98ff4c (final commit)	Solidity

Project Overview

This document describes the manual code review findings of **Polygon Vault Bridge V1**. The main focus of the review was [PR#36](#). The following contract list is included in our scope:

```
src/etc/InitializationCounterUpgradeable.sol
src/etc/Versioned.sol
src/primary-chain/MigrationManager.sol
src/primary-chain/VaultBridgeToken.sol
src/primary-chain/VaultBridgeTokenInitializer.sol
src/primary-chain/VaultBridgeTokenPart2.sol
src/primary-chain/ethereum/GenericVaultBridgeToken.sol
src/primary-chain/ethereum/vbETH/VbETH.sol
src/secondary-chain/CustomToken.sol
src/secondary-chain/CustomTokenWethExtension.sol
src/secondary-chain/NativeConverter.sol
src/secondary-chain/agglayer/CustomTokenAgglayer.sol
src/secondary-chain/agglayer/GenericCustomTokenAgglayer.sol
src/secondary-chain/agglayer/GenericNativeConverterAgglayer.sol
src/secondary-chain/agglayer/NativeConverterAgglayer.sol
src/secondary-chain/agglayer/vbETH/WethAgglayer.sol
src/secondary-chain/agglayer/vbETH/WethNativeConverterAgglayer.sol
```



```
src/secondary-chain/agglayer/vbUSDC/bridged-usdc-standard/VbUsdcNativeConverterAgglayerBridgedUsdcStandard.sol
src/secondary-chain/polygon/CustomTokenPolygon.sol
src/secondary-chain/polygon/GenericCustomTokenPolygon.sol
src/secondary-chain/wormhole/CustomTokenWormhole.sol
src/secondary-chain/wormhole/GenericCustomTokenWormhole.sol
src/secondary-chain/wormhole/vbETH/WethWormhole.sol
```

The work was undertaken from **September 22, 2025**, to **October 14, 2025**. During this time, Certora's security researchers performed a manual audit of all the Solidity contracts and discovered several bugs in the codebase, which are summarized in the subsequent section.

Protocol Overview

Vault Bridge enables chains and apps to generate native yield on TVL by putting bridged assets to work.

The protocol is comprised of:

- One Primary Chain
 - Vault Bridge Token
 - Migration Manager
- Many Secondary Chains
 - Custom Token
 - Native Converter

Select assets are bridged from Primary Chain to Secondary Chain. These assets are deposited into Vault Bridge Token contract on Primary Chain, which mints and bridges vbToken to Secondary Chain. Deposited assets are used to generate yield on Primary Chain, while bridged vbTokens are used in DeFi on Secondary Chain. Generated yield gets distributed to chains and apps participating in the revenue sharing program.

Native Converter contract can be deployed on Secondary Chain to enable acquisition of vbToken on Secondary Chain without having to bridge from Primary Chain. Accumulated backing in Native Converter on Secondary Chain gets migrated to Primary Chain and deposited into Vault Bridge Token contract.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	3	2	2
Low	2	2	-
Informational	8	7	5
Total	13	11	7

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
M-01	<code>NativeConverter</code> is not reconfigurable, breaking upgrade path	Medium	Fixed
M-02	<code>setCustomToken</code> function is vulnerable to griefing attack	Medium	Fixed
M-03	Flawed migratable backing calculation prevents fund migration	Medium	Acknowledged
L-01	<code>setYieldVault</code> can lead to temporary insolvency and orphaned assets	Low	Acknowledged
L-02	Deposit DoS risk if USDT enables transfer fees	Low	Acknowledged
I-01	Inconsistent validation on <code>destinationNetworkId</code> in <code>VbETH</code>	Informational	Fixed
I-02	Permissionless <code>reinitializer2</code> functions are vulnerable to front-running	Informational	Acknowledged
I-03	Incorrect rounding in the calculation of <code>migratableGasBacking</code>	Informational	Fixed
I-04	Missing Events for <code>setBridge</code> and <code>setNativeConverter</code> Functions	Informational	Acknowledged
I-05	<code>CustomTokenWethExtension</code> lacks <code>receive/fallback</code> — not WETH9-compatible	Informational	Fixed
I-06	Unused Error — <code>CannotWrapGasToken()</code>	Informational	Fixed

I-07	Typos	Informational	Acknowledged
I-08	README missing a step for Wormhole integration	Informational	Fixed

Medium Severity Issues

M-01. **NativeConverter** is not reconfigurable, breaking upgrade path

Severity: **Medium**

Impact: **Low**

Likelihood: **High**

Files: [NativeConverter.sol](#)

Status: Fixed

Description:

The protocol's [ReadMe](#) outlines a multi-step strategic plan for the adoption of native USDC by Circle. This plan explicitly requires a step to *"reinitialize Native Converter to convert bridge-wrapped USDC to vbUSDC Custom Token."* However, this step is impossible to execute with the current contract implementation.

The **NativeConverter** contract lacks any administrative function to update the **customToken** address after it has been set in the `__NativeConverter_init1` function. The **customToken** state variable is set only once during initialization and cannot be modified thereafter.

This oversight breaks the documented upgrade path. As a result, the protocol is locked into its initial configuration, making the strategic plan to transition assets and have Circle take over the **FiatTokenV2_2** non-viable.

Recommendations: To enable the intended upgrade path, a new, permissioned function should be added to the **NativeConverter** contract that allows an authorized admin (`DEFAULT_ADMIN_ROLE`) to update the **customToken** address. This will allow the protocol to be reconfigured as described in the operational plan.

Customer's response: Fixed in commit [a4ea0c0](#).

Fix Review: Fix confirmed.

M-02. `setCustomToken` function is vulnerable to griefing attackSeverity: **Medium**Impact: **Medium**Likelihood: **Medium**Files: [NativeConverter.sol](#)

Status: Fixed

Description:

The `setCustomToken` function is an administrative function required to execute the protocol's upgrade path. To ensure a clean transition, it requires that all underlying assets have been migrated off the `NativeConverter`, checking this with `require($.backingOnSecondaryChain == 0)`. However, any user can call the public `convert` function at any time to deposit assets, which increases the `backingOnSecondaryChain`.

This creates a race condition where a malicious user can front-run the administrator's `setCustomToken` transaction. By watching the mempool for a call to `setCustomToken`, an attacker can submit their own transaction to `convert` a small amount of tokens. This will increase `backingOnSecondaryChain` to a non-zero value, causing the admin's subsequent `setCustomToken` transaction to revert.

This can also be a normal user converting tokens, but if his transaction goes through before the `setCustomToken` transaction then `backingOnSecondaryChain > 0`.

Furthermore, the protocol's pausing mechanism cannot be used to mitigate this. While pausing the contract correctly blocks users from calling `convert`, it also blocks the `migrateBackingToPrimaryChain` function, which the admin needs to call to drain the backing to zero. This creates a catch-22: the admin cannot drain the funds while the contract is paused, but cannot safely reconfigure the contract while it is unpaused. This allows an attacker to indefinitely block a critical protocol upgrade.

Recommendations: Remove the `whenNotPaused` modifier from the `migrateBackingToPrimaryChain` function. This would allow the admin to pause the contract



(blocking user deposits) and then safely drain the remaining backing to zero before calling `setCustomToken`.

Customer's response: Fixed in commit [1ed7791](#) and [4a96b36](#).

Fix Review: Fix confirmed.

M-03. Flawed migratable backing calculation prevents fund migration

Severity: **Medium**

Impact: **Low**

Likelihood: **High**

Files:

[WethNativeConverterAgglayer.sol#L111](#)

Status: Acknowledged

Description:

The logic for calculating the `migratableGasBacking` in the `WethNativeConverterAgglayer` is flawed, which can prevent the `MIGRATOR_ROLE` from moving any funds to the primary chain, effectively trapping liquidity on the secondary chain. Additionally the `migratableBacking` calculations are also wrong and will result in a much larger value than expected.

Let's look at both of these cases:

1. Incorrect Use of `totalSupply` for Dual-Asset Backing

The `WethAgglayer` ecosystem is unique in that the `customToken` is backed by two different assets in two different locations: WETH held in the `WethNativeConverterAgglayer` and native ETH held in the `WethAgglayer` (`gasBackingOnSecondaryChain`).

However, the functions that calculate the non-migratable portion for *each* of these assets incorrectly use the `customToken().totalSupply()` as the basis for their calculation. This is incorrect because the `totalSupply` represents the combined value of *both* backing assets. This leads to a situation where the reserve requirement for each asset is calculated against the total, causing the system to reserve far more than intended.

Example:

- Assume `totalSupply()` is 100, backed by 50 WETH and 50 native ETH.
- Assume `nonMigratableBackingPercentage` = 50% and `nonMigratableGasBackingPercentage` = 50%.

- The `migratableBacking` calculation for WETH will reserve $50\% * 100 = 50$ WETH, leaving 0 migratable.
- The `migratableGasBacking` calculation for native ETH will reserve $50\% * 100 = 50$ ETH, leaving 0 migratable.

The intended behavior of allowing 25 WETH and 25 ETH to be migrated is completely blocked.

2. `totalSupply` is the Wrong Metric for Local Backing

Furthermore, using `totalSupply()` is fundamentally incorrect because it includes tokens that were minted by the bridge to represent assets already on the Primary Chain.

For example, if `totalSupply == 100` and the bridge minted 30 of those, then only 70 are actually backed by our contracts.

Given that, taking the `nonMigratablePercentage` from `totalSupply` is wrong.

Recommendations: The migratable backing calculations must be changed to be based on each contract's local backing, not the global `totalSupply`.

Customer's response: Design choice. `nonMigratable` variables tell how much should be backed by GAS in vbGAS or bwTOK in vbTOK Native Converter, relative to vbGAS/vbTOK total supply. In this example, if the percentages are set to 50%, that means 50% of the total supply should remain backed in those contracts. (i.e., non-migratable)

Low Severity Issues

L-01. `setYieldVault` can lead to temporary insolvency and orphaned assets

Severity: **Low**

Impact: **Low**

Likelihood: **Low**

Files:
[VaultBridgeTokenPart2.sol#L308-L325](#)

Status: Acknowledged

Description:

The `setYieldVault` function allows an admin (`DEFAULT_ADMIN_ROLE`) to replace the address of the `yieldVault`. However, it does not validate that the `VaultBridgeToken`'s share balance in the old `yieldVault` is zero before proceeding.

If an admin sets a new `yieldVault` while the contract still holds shares in the old one, those shares and their underlying assets become "orphaned." The protocol's accounting functions, such as `stakedAssets()` and `totalAssets()`, will immediately stop including these orphaned assets, as they only query the new `yieldVault`. This causes the `vbToken` to become undercollateralized, breaking the 1:1 backing guarantee and making the protocol insolvent.

While a competent and honest admin could theoretically recover the funds by manually setting the vault back to the old address, draining the funds, and then setting it to the new address again, this still harms the protocol's reputation.

Recommendations: The `setYieldVault` function must be modified to ensure the contract has zero shares in the current `yieldVault` before it can be replaced. This forces the admin to follow the safe procedure of calling `drainYieldVault` first.

Customer's response: Acknowledged. Depending on `yieldVaults` liquidity, it may not be possible to completely drain it. Therefore, we do not check if a VB's yield vault shares balance is zero.

L-02. Deposit DoS risk if USDT enables transfer fees

Severity: **Low**

Impact: **Low**

Likelihood: **Low**

Files: [VaultBridgeToken.sol#L1301-L1316](#)

Status: Acknowledged

Description:

Currently, `_receiveUnderlyingToken` transfers tokens from the user to the contract during the deposit flow and asserts that the actual received amount equals the value parameter.

JavaScript

```
function _receiveUnderlyingToken(address from, uint256 value) internal {
    uint256 balanceBefore = $.underlyingToken.balanceOf(address(this));

    $.underlyingToken.safeTransferFrom(from, address(this), value);
    uint256 receivedValue = $.underlyingToken.balanceOf(address(this)) - balanceBefore;

    require(receivedValue == value, InsufficientUnderlyingTokenReceived(receivedValue,
value));
}
```

While this works for standard ERC-20 tokens today, [USDT](#) exposes a configurable `basisPointRate` (currently set to zero). If that fee parameter is changed in the future, `receivedValue` could be less than `value`, causing deposits to revert and rendering the `vbUSDT` contract unusable.

Recommendations: Make the `vbUSDT` transfer-receipt logic tolerant of potential transfer fees while preventing unexpected losses as described in [L-02](#) of this audit report.

Customer's response: Acknowledged. If USDT turns fees on, it would break many contracts. unlikely to happen. There used to be a fee calculating mechanism in VB and it was removed in favor of a strict check.

Informational Issues

I-01. Inconsistent validation on `destinationNetworkId` in `VbETH`

Description:

A validation inconsistency exists in the `VbEth::depositGasTokenAndBridge` function. Unlike other similar functions in the protocol, such as `VaultBridgeToken::depositAndBridge` and `NativeConverter::deconvertAndBridge`, this function fails to check that the `destinationNetworkId` is different from the current chain's `agglayerId`.

JavaScript

```
/// @dev deposit ETH to get vbETH and bridge to an L2
function depositGasTokenAndBridge(
    address destinationAddress,
    uint32 destinationNetworkId,
    bool forceUpdateGlobalExitRoot
) external payable whenNotPaused nonReentrant returns (uint256 shares) {
    (shares,) = _depositUsingCustomReceivingFunction(
        _receiveUnderlyingTokenViaMsgValue,
        msg.value,
        destinationNetworkId,
        destinationAddress,
        forceUpdateGlobalExitRoot,
        0
    );
}
```

While this omission does not lead to a loss of funds, it creates inconsistent behavior. If a user calls `depositGasTokenAndBridge` with the `destinationNetworkId` set to the current chain's ID, the transaction will not revert. Instead, the internal `_depositUsingCustomReceivingFunction` will execute its `else` branch, simply minting `vbETH` to the receiver on the local chain without performing any bridge operation.

Recommendations: To ensure consistent behavior and logical correctness across all bridging functions, the `require(destinationNetworkId != $.agglayerId, InvalidDestinationNetworkId())` check should be added to the `VbEth::depositGasTokenAndBridge` function.

Customer's response: Fixed in commit [a57b8e5](#).

Fix Review: Fix confirmed.

I-02. Permissionless `reinitializer2` functions are vulnerable to front-running

Description:

The `reinitialize2` function in the `MigrationManager` contract is a public function that is not protected by any access control modifiers, such as `onlyRole(DEFAULT_ADMIN_ROLE)`. This function is intended to be called by the administrator as part of a multi-step upgrade process to set the `_wrappedGasToken` address.

Because the function is permissionless, it creates a race condition during a contract upgrade. An attacker can monitor the blockchain for the `upgradeTo` transaction. After the upgrade is complete but before the legitimate administrator calls `reinitialize2`, the attacker can front-run them and call `reinitialize2` with a malicious or incorrect address for the `wrappedGasToken_`.

Basically this finding is just to state that when upgrading the contracts the `reinitialize2` functions should be encoded within the upgrade call. This is valid for all of the contracts not just the Migration Manager

Recommendations: When performing an upgrade, the call to `reinitialize2` must be atomically bundled with the upgrade itself using `upgradeToAndCall`. This ensures that no other transaction can be executed between the upgrade and the reinitialization, preventing the front-running attack.

Customer's response: Acknowledged. All deployments/upgrades will use the new `reinitialize` function, which self-calls all reinitializes. There are already some deployments that do not have `InitializationCounterUpgradeable`, and those will be upgraded by passing

`reinitialize2` / `reinitialize3` data as the parameter for `upgradeAndCall`. `reinitializeX` functions are planned to be restricted to `onlySelf` in the next version, when all deployments will have `InitializationCounterUpgradeable`.

I-03. Incorrect rounding in the calculation of `migratableGasBacking`

Description:

`NativeConverter.migratableBacking()` computes the nonMigratable backing using `Math.mulDiv(..., Math.Rounding.Ceil)` (rounding **up**) to conservatively preserve the non-migratable portion.

JavaScript

```
uint256 nonMigratableBacking = _convertToAssets(  
    Math.mulDiv(customToken().totalSupply(), $.nonMigratableBackingPercentage, 1e18,  
    Math.Rounding.Ceil)  
);
```

However `WethNativeConverterAgglayer.migratableGasBacking()` uses `Math.mulDiv` with the default (floor) rounding, which can produce an off-by-one result.

JavaScript

```
uint256 nonMigratableGasBacking =  
    _convertToAssets(Math.mulDiv(customToken().totalSupply(),  
    $.nonMigratableGasBackingPercentage, 1e18));
```

Recommendations: Make the rounding behaviour consistent with `NativeConverter.migratableBacking()` by using `Math.Rounding.Ceil` when calculating the nonMigratableGasBacking

Customer's response: Fixed in commit [5bbd6fc](#).

Fix Review: Fix confirmed.

I-04. Missing Events for **setBridge** and **setNativeConverter** Functions

Description:

In `CustomToken.sol`, the administrative functions [setBridge](#) and [setNativeConverter](#) update critical contract dependencies without emitting any events. The absence of events makes it difficult to track configuration changes on-chain.

Recommendations: Emit dedicated events inside **setBridge** and **setNativeConverter**.

Customer's response: Acknowledged. Those functions are included for special cases when the bridge and/or Native Converter cannot be set immediately, and can be used only once (or never if already set).

I-05. **CustomTokenWethExtension** lacks **receive/fallback** — not WETH9-compatible

Description:

CustomTokenWethExtension is designed and documented to behave like WETH9, but it does not implement a `receive()` or payable `fallback()` function that allows direct ETH deposits. As a result, plain ETH transfers to the contract revert instead of wrapping ETH as [WETH9](#) does. This creates a functional mismatch with user expectations and the stated contract purpose. Furthermore, any external contracts or integrations assuming WETH9 compatibility, for example, protocols expecting ETH transfers to trigger deposit behavior, will fail when interacting with this contract.

Recommendations: Implement a payable `receive()` (and optional `fallback()`) function that routes to the `deposit()` method to fully mirror WETH9 behavior.

Customer's response: Fixed in commit [d04a9b1](#) and [d9e6840](#).

Fix Review: Fix confirmed.

I-06. Unused Error – `CannotWrapGasToken()`

Description: The custom error `CannotWrapGasToken()` is declared in `MigrationManager.sol` but never used in any execution path.

```
JavaScript  
error CannotWrapGasToken();
```

Recommendations: Consider removing the unused error.

Customer's response: Fixed in branch [feat/v1.0.0](#).

Fix Review: Fix confirmed.

I-07. Typos

Description: Several minor typos are present across the codebase

1. In `MigrationManager.sol`, the comment at line [263](#) incorrectly says “*bridging Custom Token “to” from Secondary Chains to Primary Chain*” — the extra “to” should be removed.
2. In `VaultBridgeToken.sol`, the documentation at line [77](#) references ERC-4246 instead of the correct standard ERC-4626.
3. In `CustomTokenWethExtension.sol`, the word “[liquidity](#)” is misspelled and should be corrected to “liquidity.”

Recommendations: Consider fixing the typos.

Customer's response: NatSpec needs to be updated in general. Will be fixed at that time.

I-08. README missing a step for Wormhole integration

Description: The Wormhole integration [README](#) documents secondary-chain setup but omits deploying/configuring the Primary-chain NTT manager. That missing step leaves the integration guide incomplete.

Recommendations: We recommend adding the step of deploying the NTT manager on the Primary Chain to make the README comprehensive and actionable.

Customer's response: Fixed in commit [59ec84e](#).

Fix Review: Fix confirmed.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.